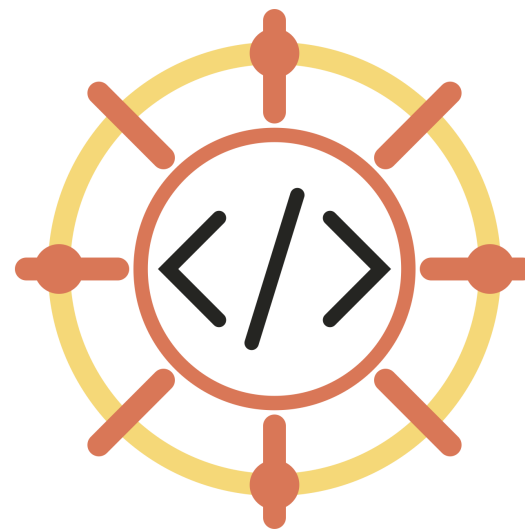


Anthropic 2 万亿 IPO 的背后

编码模型与 AGI



01

Claude 为什么强

02

编码模型如何成功

03

为什么 AGI 需要编码

两万亿，不只是在给“写代码工具”定价

从软件开发扩展到数字工作，再进入智能研发。

\$2T

IPO 目标估值

2021 成立 → 2026 IPO 目标

开发工具

写代码、修 Bug、跑测试

数字工作

对账、文件处理、跨系统操作

智能研发

运行实验、构建评测、优化基础设施

程序员是第一批用户；模型的行动能力，正在进入更多工作。

Claude 的成绩单：修软件、做工程、用工具

Claude Sonnet 5.5 · 2026.09 · 不同评测观察不同能力。

98.1%

**SWE-bench
Verified**

真实 Issue 的补丁通过测试

91.2%

SWE-bench Pro

复杂仓库任务修复

70.6%

**Terminal-Bench
4.0**

终端环境的多步任务

98.8%

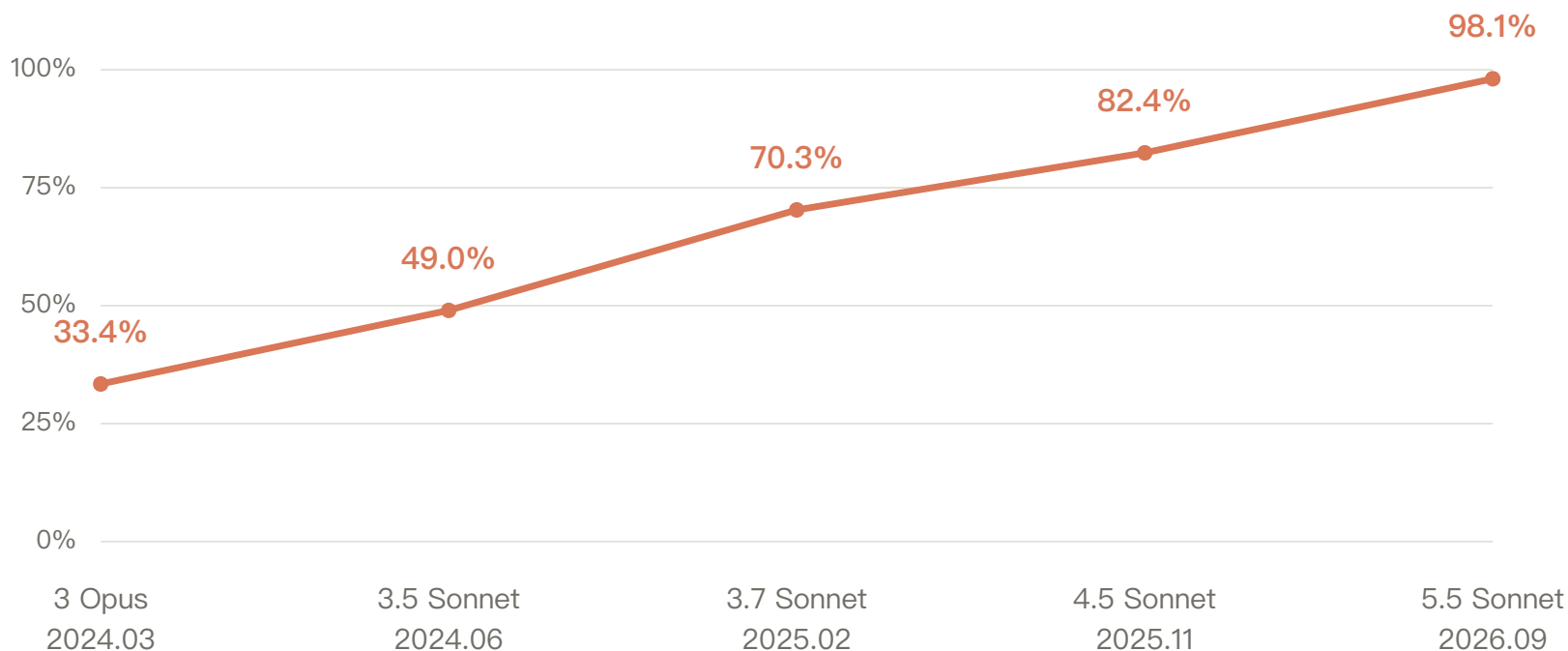
BFCL

结构化工具调用表现

它不只补全代码：搜索、修改、执行和验证，已经进入同一个任务。

从 33.4% 到 98.1%：旧标尺接近饱和

SWE-bench Verified · 仓库记录的代表模型表现。



+64.7

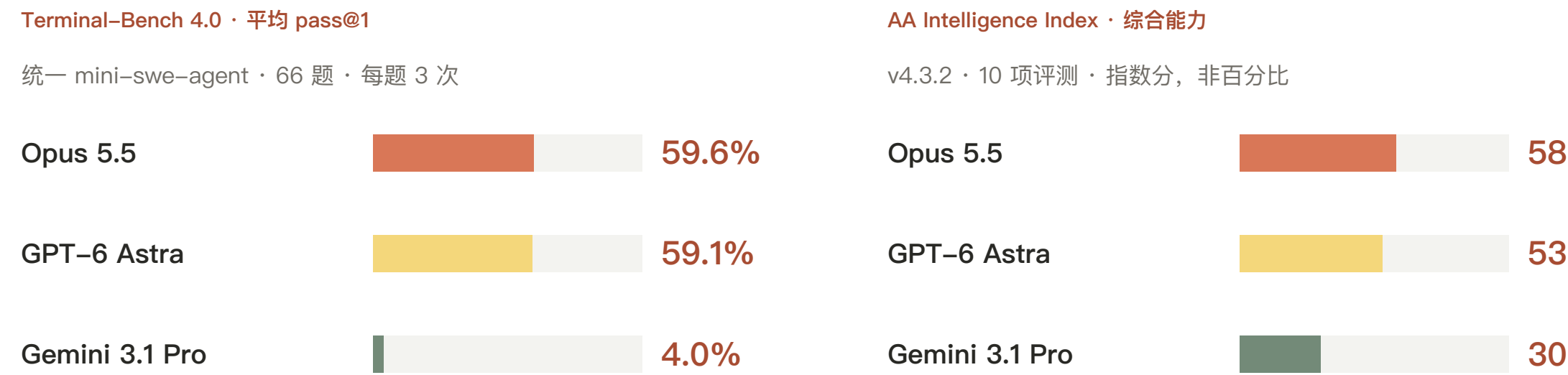
个百分点

任务变难
标尺也在升级

模型的进步，不只是答案更像人，而是更多软件问题能够被解决。

第三方对照：终端执行接近，综合能力有差异

Opus 5.5 (max / fallback) · GPT-6 Astra (max) · Gemini 3.1 Pro (Preview)



终端观测分差：仅 0.5 个百分点

未核实到完整 SWE-bench Pro 对照，改用综合指数

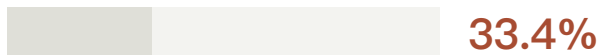
统一测试配置，才能比较分数；综合能力不等于终端任务成功率。

更强，也更便宜：能力被推向日常工作

Claude 3 Opus → Sonnet 5.5 · 仓库记录的能力与定价。

SWE-bench Verified

3 Opus



Sonnet 5.5



API 标价 · \$ / 百万 Token

输入 \$15 → \$2

输出 \$75 → \$10

输入标价下降 86.7%

当任务能力提升、调用门槛下降，更多小任务开始值得交给 Agent。

0.5 美元怎样算出来？看一次任务的账单

Sonnet 5.5 定价算例；展示费用结构，不代表所有 Bug 的实际成本。

用量假设	标价 / 百万 Token	费用	\$0.50 本次算例 模型调用费
未缓存输入 5 万	\$2.00	\$0.10	
缓存读取 100 万	\$0.20	\$0.20	
输出 2 万	\$10.00	\$0.20	

对比人力时，要比较“通过验收的任务”，不只比较一次模型输出。

反复读取相同上下文，缓存把价格压到十分之一

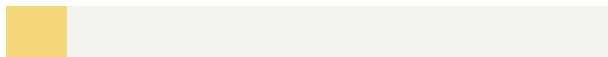
Sonnet 5.5 · 相同前缀的缓存读取；不是整张账单一律打一折。

普通输入



2.0 美元

缓存读取



0.2 美元

20 轮 × 10 万 Token

\$4.00 → \$0.40

相同前缀读取的费用算例

长任务反复使用代码背景与执行历史，缓存让连续循环更容易负担。

终端原生：同一个任务里，查、改、测、再修复

例：支付请求偶发失败，定位重试逻辑并准备补丁。

一次目标 → 多步动作

搜索	<code>rg "retry timeout" src/</code>
读取	查看支付调用与异常分支
修改	调整可重试错误与次数上限
验证	<code>pytest tests/test_payment_retry.py</code>
恢复	失败用例 → 修正 → 再运行

可检查的交付

补丁 + 回归测试

修改范围、测试结果
仍待确认的问题

人负责业务取舍与合并

CLI 的价值是动作可组合、反馈可读取，不是让 AI 拥有无限权限。

为什么不是“换个模型”就能复制体验？

工程能力来自模型判断与执行系统的共同作用。

环节	具体工程资产	解决的问题
模型判断	相关调用链、错误假设	下一步查哪里、改什么
工具接口	文件读取、精确补丁、终端回显	让动作可执行、失败可定位
上下文	CLAUDE.md、相关函数、最近报错	保留任务需要的信息
持续状态	功能清单、进度文件、Git、测试	中断后可恢复、下一会话可接班

从“回答正确”到“交付可靠”，还隔着接口、状态与验证。

200 万字的容量，不等于找对这次任务的证据

修复支付重试：当前上下文需要的是关联信息，而不是全部文件。

全量灌入

支付 + 库存 + UI + 营销 + 配置



按问题探索

测试入口 → 复现条件

支付调用 → retry 分支

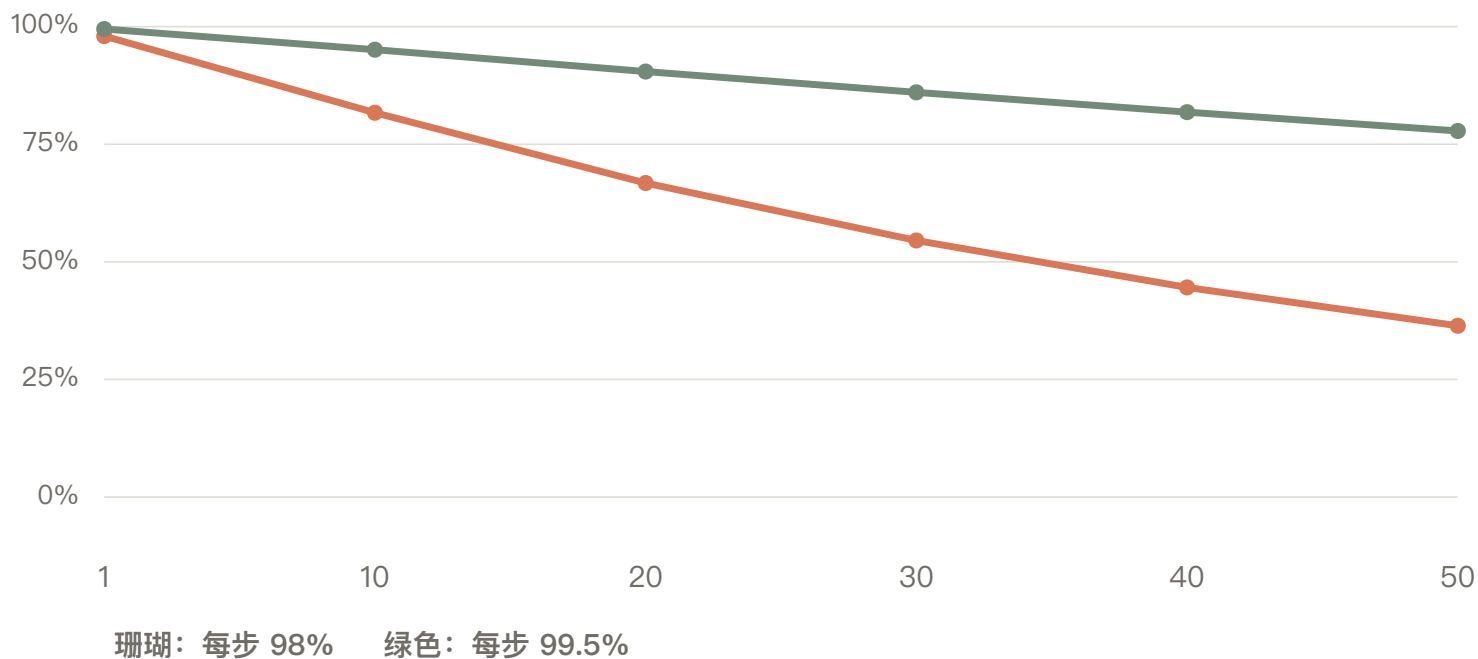
异常类型 → 超时配置

失败用例 → 修改后的验证

长窗口提供空间；主动检索决定哪些信息值得进入工作记忆。

每步只差 1.5 个百分点，50 步之后差多少？

简化模型：每步独立、可靠性恒定、失败不可恢复。



36.4%

→ 77.8%

50 步全部成功概率

真实体验的关键：少犯错、出错能恢复、少让人接管。

代码训练场有任务、有环境，也有自动裁判

从人类逐条评价，走向大规模执行与评分。

500

Verified

人工校验的 Issue 任务

642

Pro V2

离线执行的复杂仓库任务

66

Terminal-Bench 4.0

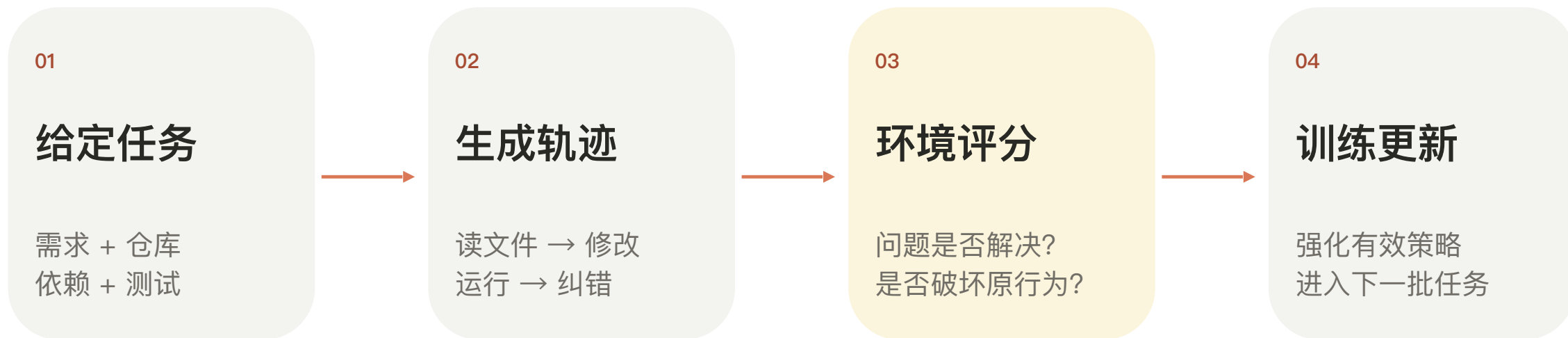
校准资源与超时的终端任务

代码库 + 容器环境 + 测试 / 验证器 → 可重复的执行反馈

编码的优势：大量试错能够运行、能够评分，也能够进入下一轮改进。

模型学习的，不只是一份正确补丁，而是整段解题轨迹

codex-1 披露：用真实编码任务强化学习，学习修改与反复测试。



执行时修好这一题；训练时让下一批题的解题策略变得更好。

任务不是“写一段 Nginx 配置”，而是满足四项验收

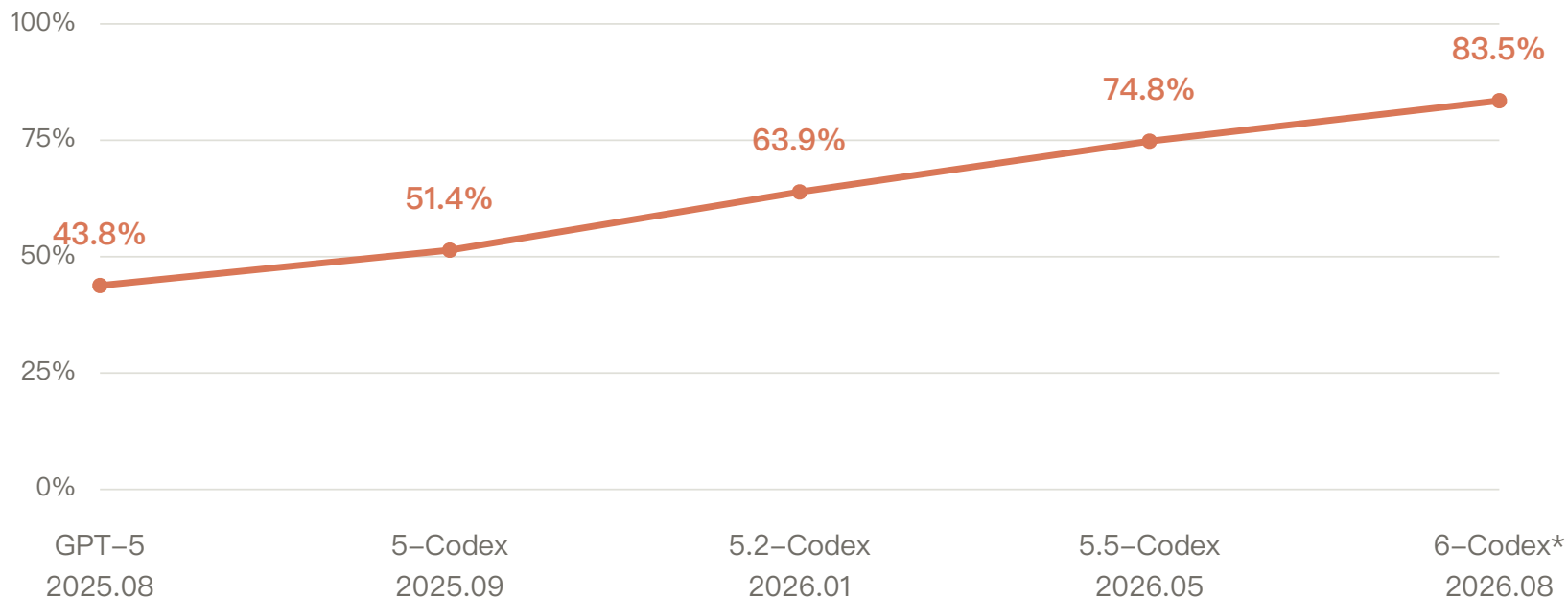
终端工程场景：证书认证、限流、签名与服务运行。

要求	验证输入	预期结果
客户端证书	有效 / 无 / 错误证书	仅有效证书可访问
请求限流	5 次/秒 与 20 次/秒	超额请求返回 HTTP 429
请求签名	请求体 + 固定密钥	上游收到正确 SHA256 HMAC
服务状态	启动、端口、配置重载	8443 监听、重载后规则生效

从“生成像样代码”到“系统行为达标”，中间有可执行的验收。

OpenAI 的编码专项迭代，也在持续爬坡

SWE-bench Pro · Codex 模型演进；* GPT-6-Codex 为预览。



43.8%

→ 83.5%

同一系列持续迭代

巨头争夺的不只是聊天入口，而是可以反复被委托的执行能力。

普通员工的任务，也包含数据结构、循环与规则

例：订单明细与银行流水对账；保留差异证据，不直接修改账目。



输出：异常清单 + 原始行号 + 核对依据

Agent 编写清洗与匹配脚本，人核对业务规则与异常解释。

新需求不必都先开发一个新产品

读接口、生成脚本、组合工具，临时补齐系统之间的缺口。

目标：对账并准备异常报告

```
orders = load("orders.xlsx")
bank = load("bank.csv")
rows = reconcile/orders, bank)
report = attach_evidence(rows)
save(report, "exceptions.xlsx")
```

CLI：执行脚本

文件处理、计算、运行与检查

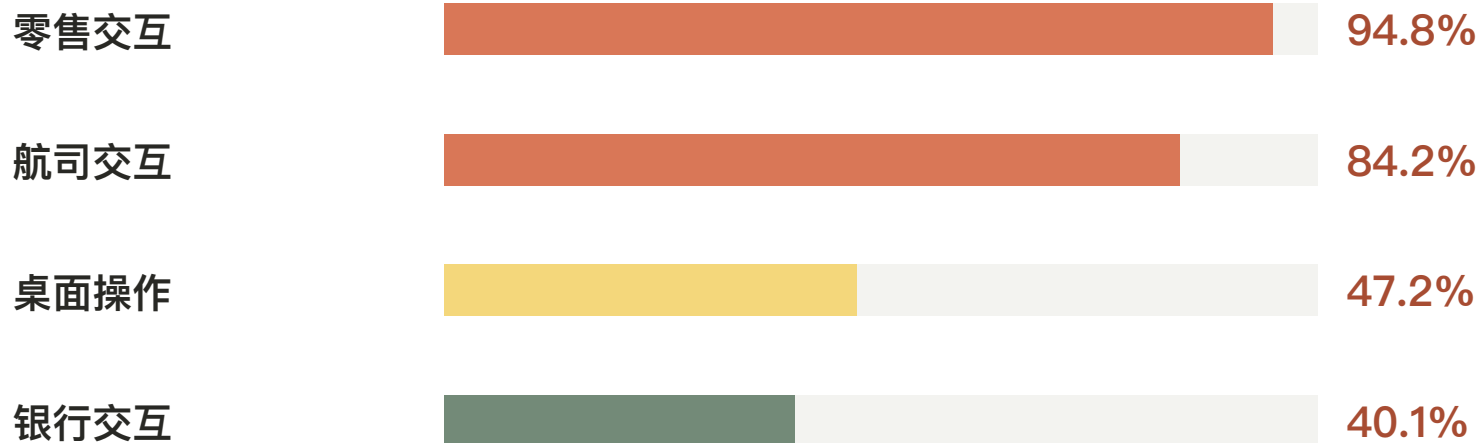
MCP：接入业务

获批的数据与办公系统

Skills 把有效流程、脚本与验收方式保存下来，让经验能够复用。

数字任务先走得更远，不同领域仍然很不均衡

Sonnet 5.5 · 业务交互与桌面操作的评测切面。



代码打开
行动空间

跨任务泛化
长期学习与业务判断
仍要继续进步

编码是通用行动的重要底座；它不是所有领域已经自动化的证明。

26% 的研发工作，已经达到 AI 主导

更强的模型，不只被卖给用户，也开始进入下一代模型的研发。

26%

AI 主导

高层目标输入 · 人类监督

>90%

至少 AI 协作

多数研发工作已包含 AI

任务级别

AL3 · 协作

AL4 · 主导

AL5 · 完全自主

AL4 是协作及以上的一部分；完全自主仍未在所测任务类别中报告。

AI 已经从研究成果，变成研究能力的一部分。

它进入研发的方式，是代码、实验与检查工具

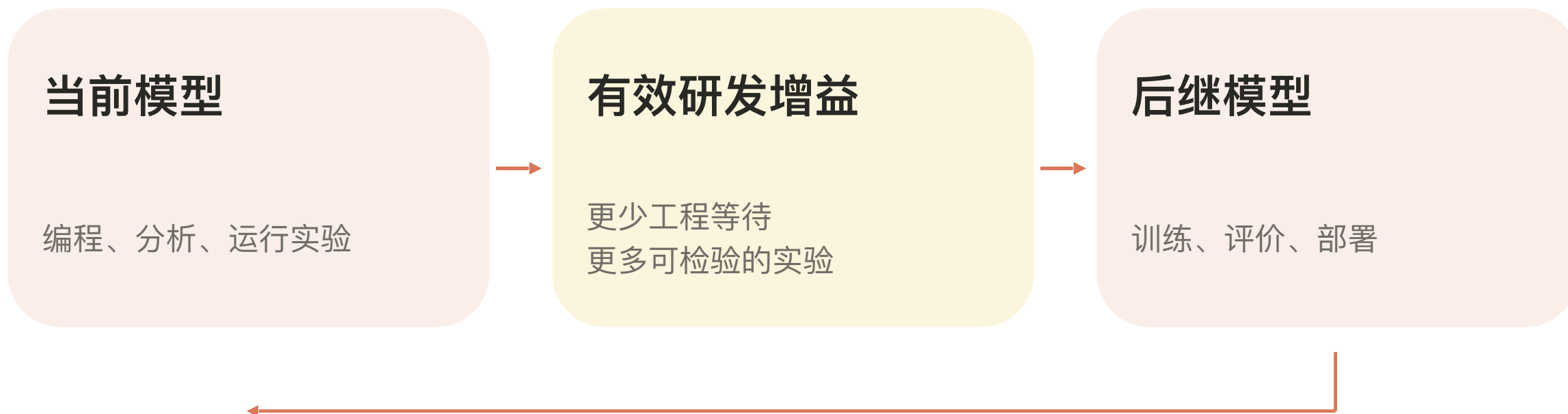
四类工作，把研究想法更快送到可以验证的环节。

研发环节	可交付的工程产物	需要验证的结果
训练基础设施	诊断日志、内核补丁、性能脚本	吞吐、稳定性、数值精度
数据与评测	候选任务、评分器、检查脚本	泛化、偏差、是否可作弊
实验执行	实验配置、运行脚本、对照表	增益是否可复现
安全研究	红队样本、复现用例、风险报告	失败机制与部署风险

研发增益要看有效实验与可靠结论，不只看生成代码量。

当前智能参与研发，后继智能再进入下一轮

RSI：改进工具的能力，也开始被工具自身放大。



最震撼的不是机器替我们工作，而是参与制造下一代更会工作的机器。

把业务经验变成可执行能力，而不只是个人熟练度

先定义目标与规则，再委托执行；用产物和证据判断是否完成。

岗位	可以交给 Agent 的产物	人应握住的判断
运营 / 市场	指标计算、交叉分析、异常报告	问题是否值得问、指标是否正确
财务 / HR	对账清单、材料结构化、来源索引	规则、例外、金额与敏感信息
产品 / 研发	可运行原型、补丁、回归测试	需求价值、架构与上线责任

对公司：重新审视排期、交接和等待，释放过去做不起的任务。